FROG - Opening the Next Era of Graphical System Programming

From GO HW to an open, inspectable, hardware-agnostic and compiler-agnostic foundation for the AI era

FROG

Free Open Graphical Language

Whitepaper

GRAIPHIC

Table des matières

Versioning 3
Preface - How This Whitepaper Fits into the Graiphic Program 4
Executive Summary 6
1. Why GO HW Required a Deeper Open Foundation 7
2. LabVIEW Proved the Paradigm. FROG Opens It. 8
3. The Missing Category: Graphical Dataflow as Language Infrastructure 9
4. What Dataflow Programming Changes 10
5. The AI-Generated Code Crisis: Speed Without Inspectability 11
6. Open Source Signals: AI Is Useful, but Ungoverned AI Is Not Acceptable 12
7. FROG's Answer: Graphical, Open, Inspectable, Human-Governed Code 13
8. The FROG Stack: .frog, Meaning, FIR, Lowering, Backend Contracts 13
9. Open FIR: The Modularity Layer That Changes Everything 15
10. Runtime and Compiler Complementarity 16
11. Runtime Families, Hardware Bridges, and Connectivity 17
12. Modern Front Panels and Runtime UI 18
13. Remote Front Panels, Observability, and Distributed Runtime Models 20
14. Current Public Proof in the Repository 21
15. What Is Proven, What Is Bounded, What Is Roadmap 22
16. Graiphic's Platform Strategy: Open Standard, Exclusive IDE, Runtime Families, Marketplace 22
17. Technology Surprise and Strategic Consequences 23
18. Path Forward 24
Conclusion 25
Appendix A - Technical Definitions 27
Appendix B - Source and Evidence Trail 28

Versioning

Version	Date	Author	Organization	Change Description
1.0	2026/04/24	Youssef Menjour	Graiphic	Strategic refinement: GO HW framed as continued evolution, stronger program preface, runtime family economics, Unreal-inspired ecosystem model, modern front-panel and AI-era reviewability emphasis.

Preface - How This Whitepaper Fits into the Graiphic Program

This whitepaper is part of a coordinated series of technical documents published by Graiphic to present the full scope, structure, and ambition of its graph orchestration program. It should not be read as an isolated publication. It is the next foundation document in a trajectory that began with SOTA GO and GO HW, and that now moves deeper into the language layer itself.

The earlier GO whitepapers shared a common foundation: ONNX as a graph format, ONNX Runtime as an execution layer, and SOTA as the visual orchestration environment. FROG changes the depth of the ambition. Instead of extending a graph strategy only through an existing graphical cockpit, FROG aims to open the graphical dataflow language foundation itself.

The complete Graiphic GO Whitepaper Series is available here: https://github.com/Graiphic/GO-Whitepapers and through the GitHub Pages experience at https://graiphic.github.io/GO-Whitepapers/.

The FROG specification repository is available here: https://github.com/Graiphic/FROG.

This FROG whitepaper now belongs to the same publication ecosystem and should be read alongside the GO whitepaper series.

Whitepaper	Role in the Graiphic program	Current status
SOTA GO - The LabVIEW IDE for Graph Computing	Introduces SOTA as the operational visual environment for authoring, inspecting, training, optimizing, deploying, and orchestrating ONNX graphs inside LabVIEW.	Operational. In production. Continuous development.
GO HW - From Models to Systems	Defines the target ambition for extending graph orchestration beyond AI inference into hardware, deployment, monitoring, timing, and system-grade execution. FROG provides the open language foundation needed to pursue it without depending on a closed third-party language substrate.	Strategic/theoretical whitepaper. Development not started; planned for FROG.
GO GenAI - From Fragmentation to Orchestration	Frames generative AI as a future graph orchestration challenge involving models, tokenizers, memory, data streams, logic, and heterogeneous runtime execution.	Strategic/theoretical whitepaper. Development not started.
GO IML - From Theory to Superiority	Frames Informed Machine Learning as a future graph-native capability where physics, logic, constraints, priors, and expert knowledge participate in learning and deployment.	Strategic/theoretical whitepaper. Development not started.
FROG - Free Open Graphical Language	Moves the Graiphic vision to the open language layer: canonical .frog source, validated meaning, open FIR, lowering, backend contracts, runtime families, compiler corridors, hardware bridges, and AI-era inspectability.	Active development. Will progressively integrate SOTA and GO capability layers.

Project maturity should be read explicitly. SOTA GO is operational, in production, and still under continuous development. GO HW, GO GenAI, and GO IML are strategic/theoretical whitepapers that define future capability layers; their standalone development has not started. FROG is the active development foundation intended to integrate these capability layers progressively once the language, FIR, runtime, compiler, and hardware-bridge corridors are mature enough.

Together, the GO whitepapers describe the need for graph-based orchestration. FROG describes the next disruptive step: transforming graphical dataflow from a product-bound capability into an open language infrastructure that can support many IDEs, many runtimes, many compiler corridors, many hardware bridges, and a new ecosystem of industrial extensions.

LabVIEW proved the magic of graphical dataflow for real engineers. The GO HW whitepaper framed the need for graph-based hardware orchestration. FROG opens the language layer, the FIR layer, and the ecosystem layer so that graphical dataflow can become a free, modular, inspectable, AI-ready and hardware-agnostic industrial standard.

Executive Summary

FROG is Graiphic's answer to a hard technological conclusion: graphical system programming cannot become a broadly open, AI-ready, hardware-agnostic industrial standard if the language layer, the execution-facing intermediate representation, the runtime boundary, and the compiler boundary remain locked inside one proprietary stack.

SOTA GO established an operational graph-based AI workflow inside the SOTA environment and remains in production under continuous development. GO HW, as a strategic whitepaper, defined the next target layer: graph orchestration for hardware, deployment, monitoring, and system control. FROG moves the ambition down the stack: from a powerful graphical cockpit to an open graphical language infrastructure where those future GO HW capabilities can be implemented on stronger ground.

Current maturity is deliberately separated throughout this document: SOTA GO is the operational production layer; GO HW, GO GenAI, and GO IML are strategic whitepapers describing planned capability layers whose standalone development has not started; FROG is the active development effort that is intended to host and integrate those layers over time.

This is not a rejection of LabVIEW. It is the opposite. LabVIEW proved that graphical dataflow could become a serious industrial programming paradigm. FROG aims to give that paradigm an open, inspectable, hardware-agnostic, compiler-agnostic, and AI-ready future.

The key architectural step is the open FIR, the FROG Intermediate Representation. FIR is not intended to be a hidden implementation detail. It is the public execution-facing bridge between validated graphical program meaning and downstream runtime, compiler, hardware, observability, and validation paths. This is the modularity layer that changes the category.

FROG aims to bring C-like implementation plurality to graphical dataflow programming. The analogy is architectural, not a maturity claim. Like standardized textual languages can be implemented by multiple compilers and target many platforms, FROG aims to enable graphical dataflow to be implemented through multiple runtime families, compiler corridors, hardware bridges, and development environments.

FROG also responds to the AI-generated code crisis. Generative AI makes software faster to produce, but not automatically easier to govern. The question is no longer simply whether engineers should use AI coding agents. The real question is how humans inspect, control, validate, and take responsibility for software produced or transformed at machine speed.

FROG does not claim to magically guarantee security. Its claim is stronger and more credible: FROG improves the conditions for inspection, accountability, traceability, and human review by keeping program structure graphical, source artifacts open, semantic validation explicit, FIR inspectable, lowering visible, backend contracts defined, and runtime/compiler handoff disciplined.

Graiphic's business strategy follows the same logic. FROG as a language and specification should remain open. The economic model should live at the product and ecosystem layers: the flagship FROG IDE, professional runtimes, certified extensions, commercial support, conformance services, and a future marketplace. The direction is inspired by modern engine ecosystems: reduce friction for adoption, let teams create value, and monetize at the level of

professional tooling, commercial success, certified distribution, and ecosystem services rather than by locking the language itself.

If successful, FROG can create a technology surprise for graphical industrial programming: a shift from closed vendor product capability to open, modular, inspectable, AI-ready, hardware-agnostic language infrastructure. The result would not merely be another IDE. It would be a new category.

1. Why GO HW Required a Deeper Open Foundation

GO HW evolve and its vision remains central to Graiphic's trajectory, but it should be read as a strategic/theoretical capability layer rather than an already-started standalone development stream. The objective remains to bring graph-based orchestration to hardware, deployment, monitoring, timing, I/O, energy awareness, and system-grade execution. What changes is the foundation required to deliver that ambition responsibly.

The GO HW whitepaper showed why a graph should become more than an AI model container. It framed a capability layer where timing, monitoring, deployment structure, hardware access, energy information, and operational behavior can be expressed graphically. It is the natural continuation of the SOTA idea: one graph, one execution logic, one cockpit for human understanding.

But as soon as graph orchestration touches real hardware and sensitive systems, a deeper requirement appears: the language substrate itself must be open, inspectable, and standardizable. Graiphic cannot responsibly build an open, multi-vendor, hardware-agnostic GO HW standard on top of a closed third-party language substrate, a proprietary saved format, or a deep compiler representation controlled by another vendor.

In the LabVIEW ecosystem, the relevant deep compiler intermediate representation, including DFIR as described in public NI compiler material, is proprietary and not open. That is an understandable product reality for NI, but it is not a sufficient foundation for Graiphic's objective: an open graphical dataflow standard that can support third-party runtimes, compiler corridors, hardware vendors, and independent ecosystem growth. Graiphic also cannot base a long-term platform strategy on assumptions about a third-party product roadmap or NI's future strategic direction for LabVIEW.

FROG is the architectural answer to that constraint. It is the open language foundation that makes the GO HW ambition more durable: canonical source, validated meaning, open FIR, lowering, backend contracts, runtime bridges, compiler bridges, and ecosystem governance. GO HW defined the hardware-orchestration target; FROG provides the open substrate required to implement and generalize that target across the broader hardware industry.

The ambition did not shrink. It moved down the stack

2. LabVIEW Proved the Paradigm. FROG Opens It.

LabVIEW is one of the strongest industrial proofs that graphical dataflow can be more than a teaching concept or a prototyping convenience. It proved that engineers can build serious measurement, control, automation, and embedded systems when the programming model matches the way systems behave.

For Graiphic, LabVIEW is a successful proof-of-concept at industrial scale. It proved the paradigm. FROG tries to generalize the underlying concept beyond one product ecosystem, one proprietary compiler stack, and one hardware-vendor history.

That proof is strategically important. It shows that engineers can build serious systems when they reason in terms of front panels, block diagrams, dataflow, indicators, controls, timing, hardware interaction, and live observability. LabVIEW made software feel like an instrument.

FROG keeps that intuition, but changes the architecture. In historical graphical environments, the language, saved format, IDE, runtime, compiler pipeline, hardware targets, and commercial product boundary were tightly coupled. That integration created enormous practical advantage, but it also created a closed user ecosystem and a closed technical ecosystem. FROG is built on a different premise: keep the graphical power, but open the language substrate and the FIR bridge layer.

FROG aims to separate what historical platforms combined. The language should not be owned by one IDE. Execution should not be owned by one runtime. Compilation should not be owned by one backend. Hardware access should not be owned by one vendor. The official product can be excellent and exclusive, while the language foundation remains open.

For a LabVIEW reader, this should feel familiar and new at the same time. Familiar because FROG preserves the power of dataflow, front panels, runtime interaction, and engineering visibility. New because FROG turns these ideas into an open language infrastructure with source, FIR, lowering, backend contracts, conformance, runtime families, compiler corridors, and marketplace-ready ecosystem boundaries.

FROG is not a clone of LabVIEW. It is a clean architectural generalization of the graphical dataflow idea for an era of generative AI, heterogeneous hardware, open IRs, runtime families, compiler corridors, and ecosystem marketplaces. The historical lesson is preserved; the platform model changes.

> *LabVIEW proved that graphical dataflow could become a serious industrial programming paradigm. FROG aims to give that paradigm an open, inspectable, hardware-agnostic, compiler-agnostic, and AI-ready future.*

3. The Missing Category: Graphical Dataflow as Language Infrastructure

Modern software still forces an old trade-off. Syntax-first languages can deliver determinism, performance, deep control, compiler maturity, and deployment seriousness, but they often impose high implementation friction and poor structural visibility at system scale. Accessible visual environments can reduce friction, but they often remain product-bound, runtime-bound, hardware-bound, or too shallow for serious industrial deployment.

FROG is designed to break that trade-off by creating a new paradigm: **a precise open source language artifact for machines, and a graphical dataflow interface for humans**.

At the machine level, FROG is not vague, informal, or merely visual. A FROG program is saved as an open, structured, precise .frog source artifact. That source can be versioned, parsed, generated, validated, transformed, and lowered like any serious language artifact. It gives tools, AI agents, validators, runtimes, and compilers a deterministic representation to work with.

At the human level, the same .frog source can be rendered, inspected, and edited through a graphical dataflow IDE inspired by what LabVIEW proved for industrial engineering: nodes, wires, front panels, visual structure, probes, indicators, and direct system readability. The graphical interface is not a decorative layer above the language. It is the human-scale representation of a precise open source artifact.

This is the best of both worlds.

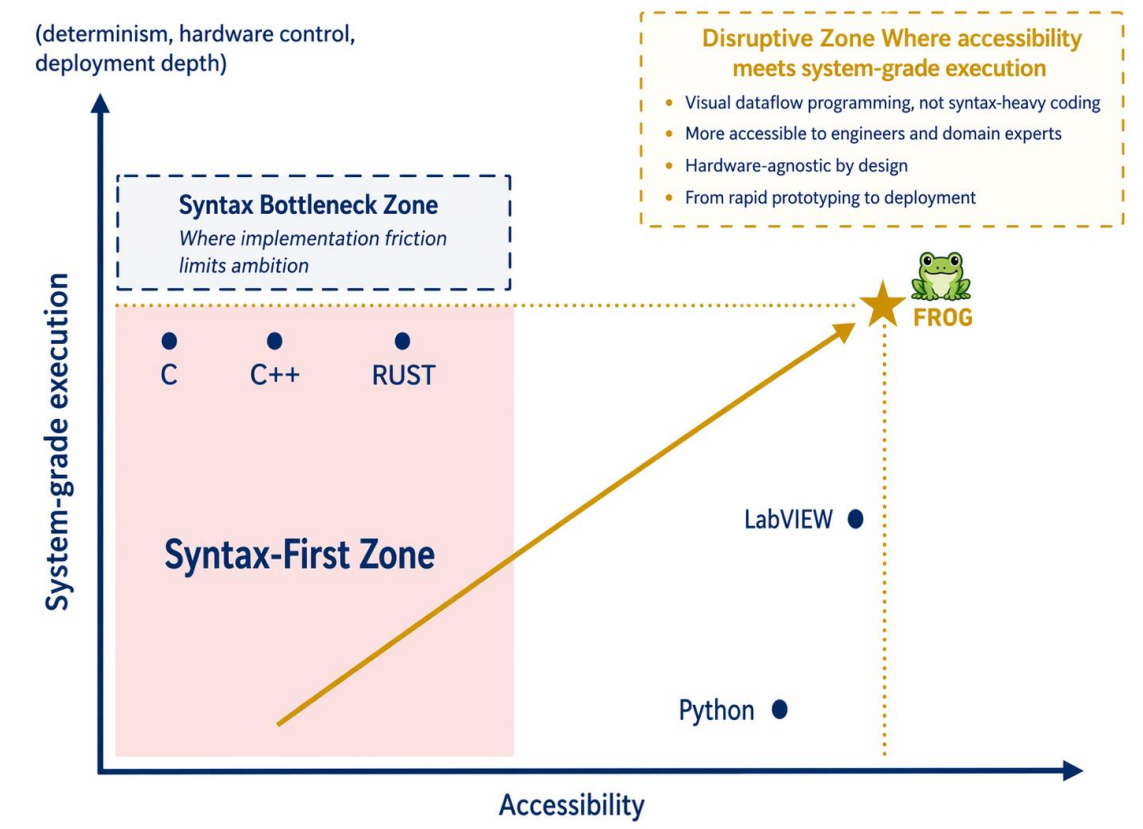

Figure 1 - FROG targets the missing zone where accessibility meets system-grade execution.

FROG keeps the rigor of language infrastructure: canonical source, validated meaning, open FIR, lowering, backend contracts, conformance, runtime families, compiler corridors, and hardware bridges. But it exposes that rigor through an accessible graphical model that engineers can inspect, understand, and operate without being trapped in syntax-heavy implementation work.

The long-term ambition is comparable in spirit to what C achieved for textual systems programming: not one compiler, not one runtime, not one hardware vendor, but a language foundation that can be implemented across many targets. FROG aims to bring that implementation plurality to graphical dataflow programming. Through open FIR-based compiler and runtime bridges, FROG is designed to become hardware-agnostic by architecture while remaining efficient on each target through the most appropriate downstream compiler, runtime, or vendor-specific execution path.

In other words, FROG does not choose between accessibility and execution seriousness. It combines them. It gives machines a precise open source format and gives humans a graphical control surface. That combination is what makes FROG more than a graphical tool: it makes FROG a candidate foundation for a new generation of open, inspectable, hardware-agnostic graphical system programming.

The gain is not only ergonomic. It is architectural. A graphical language with open source, open FIR, explicit conformance, runtime plurality, compiler plurality, and hardware bridge modularity becomes a platform for ecosystem leverage, not simply an authoring interface.

That is the category FROG wants to open: free and open graphical dataflow programming as an industrial language standard.

4. What Dataflow Programming Changes

Dataflow Programming in the AI Era: Machine-Generated Source, Human-Scale Inspection

Dataflow programming describes computation through dependencies, value movement, and explicit system structure rather than through a primarily linear sequence of instructions. A node executes when its required inputs are available. Edges make data movement visible. The graph exposes the dependency structure of the system.

This matters because real industrial systems are rarely mentally linear. They are networks of sensing, computation, control, memory, timing, communication, observation, and actuation. Textual languages can represent those systems, and they remain powerful. But the human reviewer often has to reconstruct the architecture by navigating files, functions, calls, variables, imports, modules, build rules, and runtime behavior.

At small scale, that reconstruction is manageable. At AI-generated scale, it becomes a strategic problem. Modern coding agents can produce thousands of lines quickly, and future systems may include millions of generated or transformed lines. Human control cannot depend only on reading every line in sequence.

FROG addresses this challenge by separating two complementary layers.

First, the canonical program source remains an open .frog artifact. This is essential. A FROG program is not trapped inside an opaque graphical file. It is saved in an open, structured,

machine-readable source format that can be inspected, versioned, transformed, and generated by modern AI coding tools. In that sense, .frog can become a first-class generation target in the same way that tools today generate C, C++, Python, Rust, or other syntax-first languages.

Second, that same .frog source can be rendered and inspected through a graphical dataflow interface. This is where the human advantage appears. The graphical interface turns the machine-readable source into a human-scale representation: layout, topology, colors, shapes, wire thickness, operator families, grouping, hierarchy, front panels, probes, watch points, and observability overlays. These visual dimensions do not replace correctness rules, but they make structure faster to understand and easier to review.

This is the key innovation: FROG is designed to be both machine-generable and human-inspectable.

AI tools such as Codex, Claude Code, Cursor, or future coding agents could generate valid .frog source. Engineers could then inspect the resulting system graphically, review the structure, validate the semantics, examine the FIR, check the lowering path, and decide whether the program is acceptable for execution. FROG therefore creates a controlled co-production loop between machine-speed generation and human-scale understanding.

The result is a new kind of interface for the AI era. The computer can generate structured .frog source. The human can inspect the generated system through a graphical representation designed for cognition, review, and control. FROG moves software creation from a purely computer-scale artifact toward a human-governable engineering object.

Syntax is not the enemy. Uncontrolled scale is the enemy. FROG adds the missing layer: an open, machine-readable source format paired with a graphical, structured, execution-aware interface for human inspection, validation, and responsible co-production.

5. The AI-Generated Code Crisis: Speed Without Inspectability

Generative AI is not a side topic. It is central to why FROG matters now.

AI coding agents can produce or transform large amounts of software in minutes or seconds. This is a major innovation. It creates enormous economic gain and accelerates the path from idea to result. It also creates a new problem: production can move faster than inspection.

An engineer who hand-writes a system over weeks builds a mental model while producing it. An engineer who receives a large AI-generated codebase may receive output without the same gradual understanding. The code may compile. It may even pass tests. But the organization still must know what it does, who accepts it, how it interacts with hardware, and how it will be maintained.

The real question is not “with AI or without AI”. That is already too small. The real question is: how do humans inspect, control, validate, and take responsibility for software produced or transformed at machine speed?

In sensitive industries, that question is existential. Generated software may drive machines, instruments, robots, production lines, embedded devices, energy systems, defense systems, medical systems, or infrastructure. A large opaque block of generated syntax is not enough. The industry needs a control surface.

FROG gives the AI era that control surface: graphical structure, open source artifacts, inspectable FIR, explicit semantic validation, visible lowering paths, backend contracts, observability hooks, and future IDE-level support for provenance and review.

6. Open Source Signals: AI Is Useful, but Ungoverned AI Is Not Acceptable

The open-source world is already reacting to the review and provenance problem. The response is not uniform, and that is precisely the lesson. Some communities allow AI assistance under human responsibility. Others restrict or reject AI-generated contributions because they cannot accept legal uncertainty, low-effort output, weak maintainability, or review overload.

The Linux kernel has taken a pragmatic approach. It does not impose a blanket ban on coding assistants, but it preserves human responsibility: licensing compatibility, correct attribution, human Signed-off-by, and disclosure such as Assisted-by where appropriate remain essential.

QEMU takes a stricter position. Its code provenance documentation rejects contributions believed to include or derive from AI-generated content, because the DCO requires contributors to understand and guarantee the provenance and license status of what they submit.

NetBSD, Gentoo, Servo, elementary OS, GIMP, and GNOME Shell Extensions also illustrate the same pressure from different angles: legal provenance, quality, contributor understanding, ethics, review burden, and maintainability.

curl illustrates the review-load problem. Maintainers can be overwhelmed by plausible but weak or false reports, often described as AI slop. When generation becomes cheap, review becomes the scarce resource.

The lesson is clear: refusing AI is not a long-term strategy for industry. Governing AI-generated software is.

FROG is positioned at that boundary. It does not ask industry to reject AI. It offers an architecture for making AI-assisted system creation more inspectable, more attributable, more controllable, and more compatible with human engineering responsibility.

7. FROG's Answer: Graphical, Open, Inspectable, Human-Governed Code

FROG's AI-era proposition is simple: generated software must become more inspectable, not merely more abundant.

FROG gives the AI era a better human control surface through:

- graphical structure that exposes dataflow and dependency relationships;
- canonical .frog source that remains open, structured, versionable, and machine-friendly;
- validated program meaning that separates source shape from executable meaning;
- open FIR that preserves an inspectable execution-facing representation;
- lowering boundaries and backend contracts that keep downstream execution explicit;
- observability hooks, probes, watches, and future debugging surfaces;
- conformance and versioning discipline that distinguish accepted, rejected, preserved, unsupported, and degraded behavior.

The future Graiphic FROG IDE can make AI provenance visible rather than hidden. It may distinguish AI-generated regions from human-authored or human-reviewed regions through color overlays, provenance badges, Assisted-by metadata, generated-versus-reviewed states, validation status, conformance status, review checkpoints, and human acceptance markers.

That matters because safety is not a label. It is a process. FROG does not replace engineering responsibility. It gives engineers a stronger control surface for the AI era.

8. The FROG Stack: .frog, Meaning, FIR, Lowering, Backend Contracts

The technical strength of FROG is its separation of layers. The public repository is structured as a layered language stack: Expression owns canonical source and structural validity; Language owns validated meaning; IR owns the canonical open execution-facing representation; lowering and backend contracts define downstream handoff; Profiles define optional capability families; Conformance defines public accept, reject, and preserve expectations; Versioning centralizes corpus status.

The key pipeline is:

```
.frog canonical source
  -> structural validity
  -> validated program meaning
  -> open execution-facing FIR
  -> lowering
  -> backend contract
  -> runtime family / compiler family / hardware bridge
  -> deployment target
```

This pipeline is central to the credibility of FROG. It means source, meaning, IR, lowering, and execution are not collapsed into one implementation. The language remains upstream. Execution becomes modular downstream.

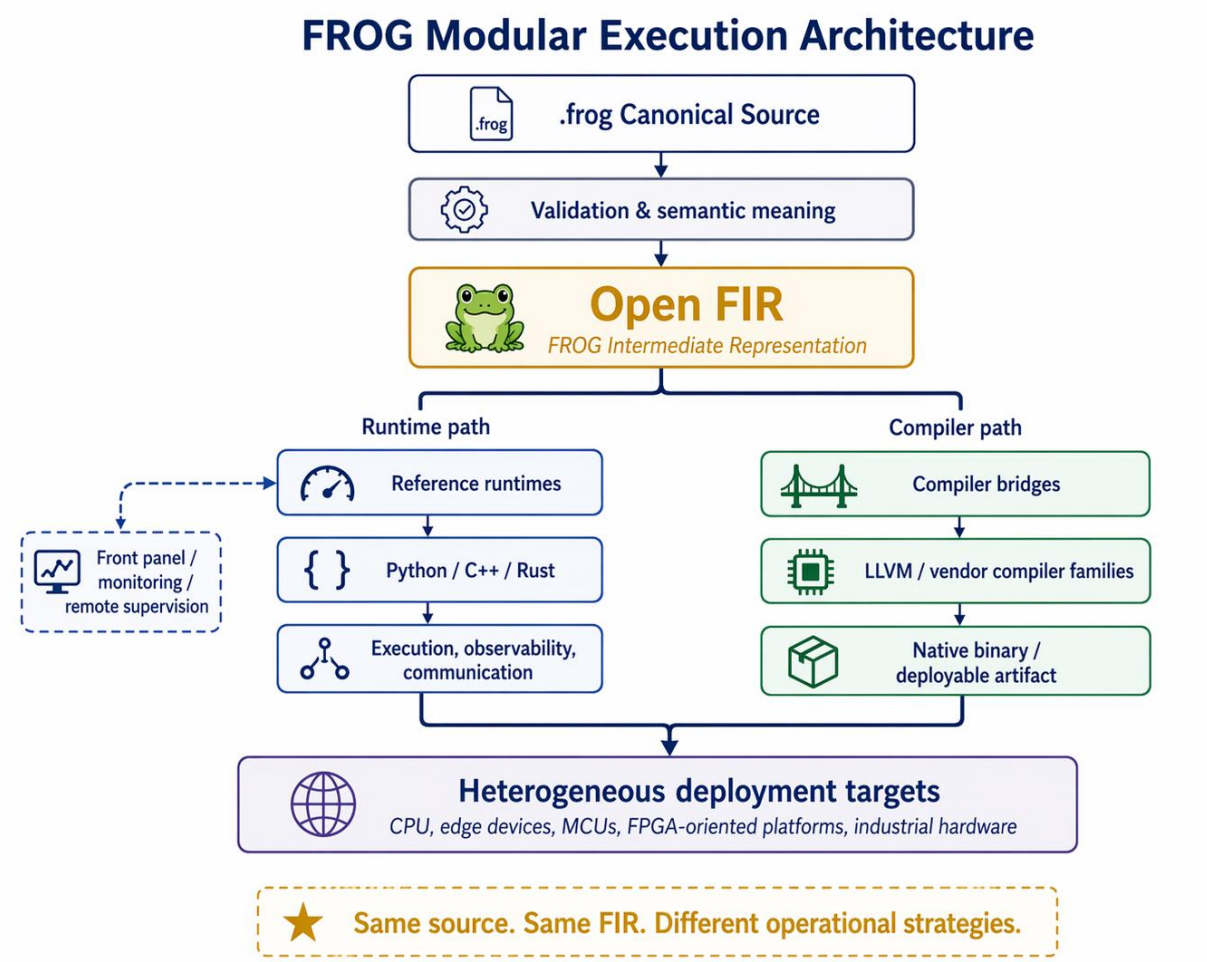

Figure 2 - FROG separates source, meaning, open FIR, runtime paths, compiler paths, and heterogeneous deployment targets.

9. Open FIR: The Modularity Layer That Changes Everything

Open FIR is the strategic center of FROG.

Without open FIR, every serious execution strategy tends to collapse into a private implementation path. With open FIR, the language can remain stable and inspectable while downstream execution remains modular.

FIR is not meant to be a hidden compiler implementation detail. It is intended to be an open, inspectable, recoverable execution-facing artifact that can support analysis, transformation, validation, observability, lowering, and backend handoff.

From the same .frog source and the same open FIR, different operational strategies can be built:

- FIR to runtime for live execution, observability, debugging, and front-panel interaction;
- FIR to compiler for native artifacts, optimization, and target specialization;
- FIR to vendor runtime for existing hardware execution ecosystems;
- FIR to hardware bridge for board, device, FPGA, SoC, or industrial target integration;
- FIR to analysis or validation tools for assurance and review;
- FIR to simulation or digital twin tools for pre-deployment verification.

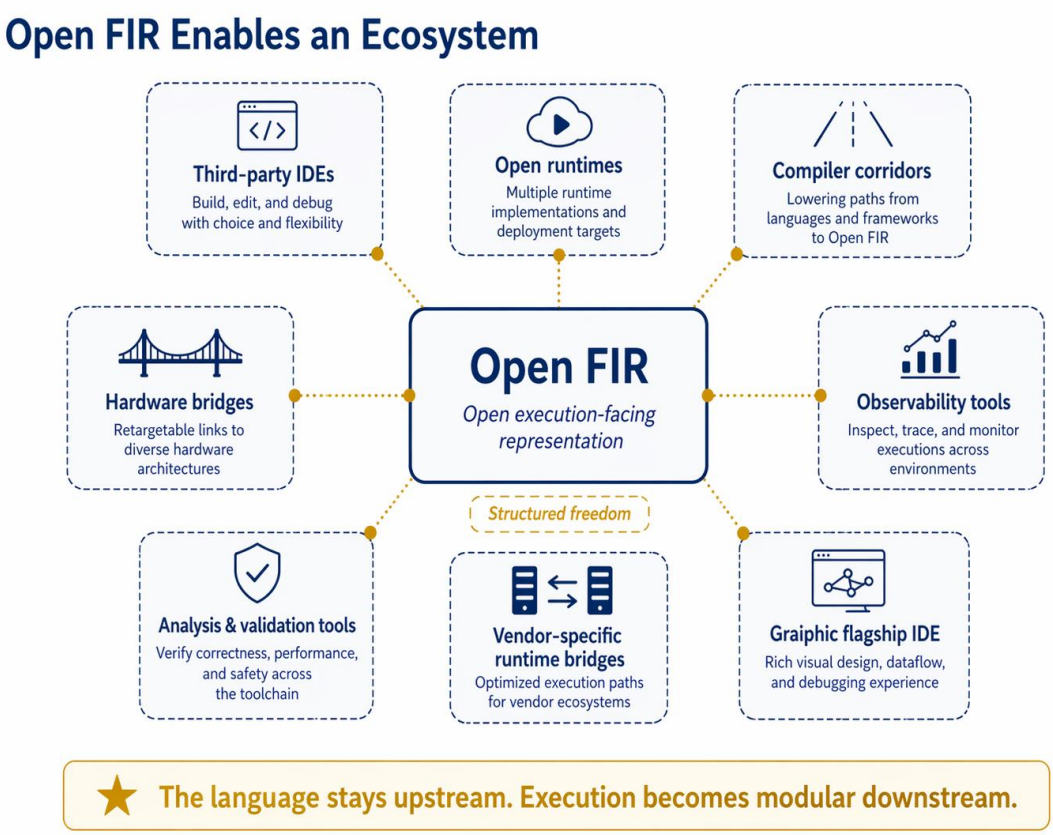

Figure 3 - Open FIR enables an ecosystem of IDEs, runtimes, compiler corridors, hardware bridges, validation tools, and Graiphic's flagship IDE.

This is the core technical moat. FROG aims to make graphical dataflow modular in the same structural sense that major textual languages became modular: one language definition, many implementations, many toolchains, many target strategies.

10. Runtime and Compiler Complementarity

FROG does not oppose runtimes and compilers. It makes them complementary downstream strategies from the same open FIR.

A compiler prepares execution. It transforms FIR or lowered forms into a target-specific executable, binary, object, bitstream-oriented input, or deployable artifact. This path is useful for native binaries, optimization, static deployment, embedded systems, reduced footprint, and target specialization.

A runtime animates execution. It provides the operational layer that executes, schedules, observes, communicates, debugs, monitors, interprets runtime UI surfaces, coordinates variables, and exposes interaction with the outside world. This path is useful for flexibility, observability, live development, remote supervision, distributed front panels, and dynamic control.

The runtime should not be confused with the language. It consumes contracts and artifacts after source, meaning, FIR, lowering, and backend handoff. It may execute directly, host a UI, communicate with a target, expose foreign-language bridges, load compiled components, or coordinate IO. But it must not become the hidden owner of FROG semantics.

The compiler should not be confused with the language either. LLVM, a vendor compiler, an FPGA flow, or a native backend may be a powerful corridor, but none of them define FROG. They are downstream consumers.

This is where FROG generalizes an idea already familiar from industrial graphical systems. NI proved that different execution contexts may need different execution strategies: development execution, real-time execution, FPGA execution, and host/target communication are not the same operational problem. FROG takes that proof and generalizes it beyond one vendor ecosystem and one product family.

In a compiled deployment, the compiler prepares an artifact: binary, object, native executable, or target-specific package. The runtime may then load, supervise, expose variables, host widgets, manage communication, provide monitoring, coordinate protocols, and make the deployed system observable. The compiler prepares the execution. The runtime makes the execution operational.

11. Runtime Families, Hardware Bridges, and Connectivity

A key FROG advantage is the ability to define runtime families adapted to different operational needs rather than forcing one heavy universal runtime everywhere.

A minimal runtime for pure operation does not need the same weight as a development runtime with debugging, front-panel interaction, live probes, trace capture, and communication variables. A microcontroller-oriented runtime does not need the same capabilities as an industrial PC runtime. An FPGA-oriented bridge does not need the same responsibilities as a Python host runtime.

FROG therefore aims to support runtime recipes such as:

Runtime recipe	Primary role	Typical capabilities
Operational runtime	Run a bounded FROG execution contract with minimal overhead.	Scheduling, state, deterministic stepping, minimal diagnostics, target IO.
Operational + observability runtime	Run while exposing controlled visibility into execution.	Metrics, probes, watches, logging, health checks, controlled indicators.
Development runtime	Support IDE-driven execution and debugging.	Breakpoints or dataflow-appropriate pause points, live front panel, inspection overlays, tracing, reload/hotswap where safe.
Distributed runtime	Separate target execution from workstation supervision.	Communication variables, protocol bridge, remote front panel, target health, remote deployment and monitoring.
Hardware-vendor runtime bridge	Expose a hardware vendor's target stack to FROG.	Runtime adapter, hardware profile, IO binding, vendor SDK interop, conformance declarations.
Compiler-backed runtime	Host or supervise compiled artifacts.	Artifact loading, lifecycle management, monitoring, side-channel control, diagnostics.

The open specification and a first family of reference runtime examples are intended to be sufficient for any serious engineering team to study the execution contract and build its own runtime if it chooses to invest in that target. Graiphic can provide reference paths in Python, C/C++, and Rust as proof recipes, but those recipes should not become a monopoly over execution.

This matters for hardware vendors and industrial ecosystems. If STMicroelectronics wants to build a FROG-compatible runtime for STM32-class devices, the architecture should allow it. If Intel, AMD, Lattice, or another FPGA, CPU, accelerator, or SoC vendor wants to expose its hardware through a FROG runtime or bridge, it should be free to do so under its own engineering responsibility and support model.

Connectivity is also part of the runtime story. In the same way LabVIEW can interact with DLLs, Python, .NET, drivers, field buses, and external services through runtime and integration

mechanisms, FROG runtimes can provide bridges to existing languages, binary interfaces, protocols, industrial buses, device drivers, cloud services, and local or remote I/O. The runtime family defines which of those capabilities exist for a given deployment posture.

The goal is not one runtime for everything. The goal is a family model: choose the runtime, compiler, bridge, communication layer, and observability level that match the use case. That creates execution efficiency, engineering economy, and better security posture. A small target should not carry the weight of a full IDE runtime. A development machine should not be limited to a minimal embedded runtime. A safety-relevant system should expose only the interactions it is designed to support.

This model transfers technical responsibility to the actors who choose to take it. A vendor that wants access to a new hardware range can implement and maintain the runtime or bridge for that range. Graiphic can provide the standard, examples, conformance direction, and marketplace channel, but the ecosystem can expand without requiring Graiphic to maintain every hardware backend in the industry.

12. Modern Front Panels and Runtime UI

FROG must also modernize the front panel.

In industrial graphical programming, the front panel is not just a convenience view. It is the human control surface: the place where operators, engineers, researchers, and maintainers observe, tune, test, validate, and interact with the running system.

Historical front panels proved that engineering software becomes more powerful when it feels observable. FROG preserves that strength while modernizing it for the AI era, heterogeneous deployment, distributed execution, and contemporary interface expectations.

A future FROG front panel can support richer widgets, provenance overlays, generated-versus-human-reviewed color states, validation status, conformance markers, runtime health, target binding, hardware profile visibility, live observability, and dataflow-aware debugging.

A key part of this modernization is the introduction of a dedicated widget package format: .wfrog.

This distinction is essential. The canonical .frog file remains the authoritative source of the program, and the diagram remains the executable authority. A .wfrog file does not replace the program source. Instead, it provides a structured way to define widget packages, widget resources, visual assets, layout assumptions, interaction surfaces, and runtime interpretation requirements.

In simple terms:

.frog defines the program.

.wfrog defines how widgets can be packaged, realized, and interpreted by compatible runtimes.

This allows widgets to be defined cleanly and consumed consistently. A button, numeric control, chart, indicator, or future industrial widget should not depend on undocumented UI behavior hidden inside one IDE. It should have a clear package structure, explicit resources,

and standardized interpretation rules so that compatible runtimes can render, bind, inspect, and interact with it predictably.

This is also where modern visual formats matter. By allowing widget visual assets to rely on scalable formats such as SVG, FROG can make front-panel components size-agnostic, easier to theme, easier to inspect, and easier to evolve over time. A widget should not be trapped in fixed-resolution legacy assumptions. It should be expressible through portable visual resources that can scale across desktop IDEs, embedded supervision panels, high-DPI displays, web-based panels, and remote monitoring environments.

The .wfrog format therefore plays a central role in FROG's front-panel modernization strategy. It makes widgets portable, inspectable, reusable, themeable, and compatible with runtime families that choose to support the relevant widget profile.

At the same time, widget realization remains separate from language truth. The widget package defines how a widget can be represented and interpreted by a runtime; it does not silently redefine the semantics of the program. Program logic remains in .frog. Validated meaning remains in the language layer. Runtime consumption remains downstream.

The modern FROG front panel should therefore become more than a LabVIEW-style UI surface. It should become an inspection cockpit for AI-assisted, distributed, hardware-aware, runtime-modular systems: a place where software structure, execution state, provenance, observability, scalable widgets, target interaction, and operator control converge.

13. Remote Front Panels, Observability, and Distributed Runtime Models

One important deployment model separates the execution target from the supervision interface.

In this model, operational logic runs on an embedded or edge target. The front panel, monitoring, control, and debugging experience remain on an engineering workstation. Communication variables or a protocol connect the two. The engineer can deploy, supervise, tune, observe, and validate without forcing the target to carry the full weight of the IDE.

For example, a future Jetson-class deployment could run FROG runtime or compiled artifacts on the target, while the workstation hosts the Graiphic FROG IDE, front panel, monitoring, and control layer. The same pattern can extend to microcontrollers, industrial PCs, FPGA-oriented platforms, or custom embedded hardware, depending on the available runtime and bridge.

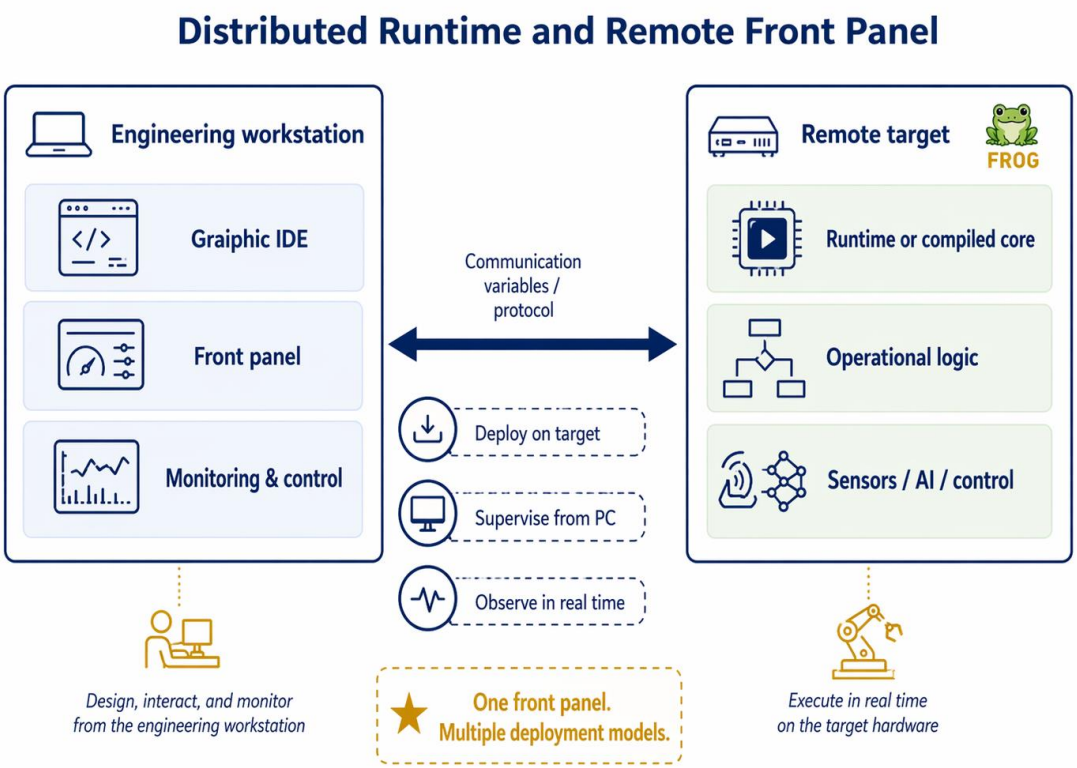

Figure 4 - Distributed runtime and remote front panel: operational logic runs on the target while engineers supervise from the workstation.

This is not just a convenience feature. It is an industrial deployment pattern. It supports development, operations, observability, debugging, commissioning, remote supervision, and safe runtime specialization.

14. Current Public Proof in the Repository

FROG is not yet a fully closed industrial platform. It should not be described as one. But it is already more than a strategy document.

The public repository defines a layered language architecture and includes bounded proof corridors. The most important current anchor is Example 05, the bounded UI accumulator. It is deliberately small, but it is serious.

Example 05 connects:

```
main.frog
 -> ui/accumulator_panel.wfrog
 -> main.fir.json
 -> main.lowering.json
 -> backend-family contract
 -> shared runtime-family acceptance
 -> Python / Rust / C-C++ runtime-family consumers
 -> first downstream LLVM proof
```

The example includes front-panel participation, widget-value participation, widget-reference participation, bounded structured control, explicit local state, public output publication, emitted backend-family handoff, shared runtime-family acceptance, multi-language runtime consumers, and a narrow LLVM-native proof corridor.

The repository is also clear about the boundary. Example 05 does not claim full generalized widget closure, full generalized runtime symmetry for all future examples, full automatic backend-family generation, or full native rendered front-panel closure. Its purpose is narrower: make one serious source-to-execution corridor visible, inspectable, executable, and stable.

This kind of proof matters for serious technical readers. It gives FROG more than a vision. It gives FROG a bounded result, a testable corridor, and a foundation for iterative closure.

15. What Is Proven, What Is Bounded, What Is Roadmap

The whitepaper should be visionary, but it must never confuse current proof with future ambition. The table below separates the current reading clearly.

Category	Current status	How to describe it credibly
Already structured in the repo	Layered architecture: Expression, Language, IR, Libraries, Profiles, IDE, Conformance, Versioning, Governance, Strategy, Reference Implementation.	A real public standard stack is already visible.
Already demonstrated	Example 05 source-to-FIR-to-lowering-to-contract-to-runtime-family corridor.	A bounded proof corridor, not a complete industrial platform.
In bounded proof	Python, Rust, and C/C++ runtime-family consumers; first LLVM-oriented proof corridor.	Use as proof of modular direction and implementability.
In active draft	Broader runtime families, widget ecosystem, front-panel generalization, richer observability and IDE behavior.	Describe as under consolidation or future closure.
Future ambition	Hardware-vendor runtimes, compiler bridge marketplace, certified extensions, third-party runtime ecosystem, Graiphic flagship FROG IDE.	Describe as platform strategy and roadmap, not current completion.

16. Graiphic's Platform Strategy: Open Standard, Exclusive IDE, Runtime Families, Marketplace

FROG's business model should not depend on locking the language. That is the strategic point. The language and specification should be open so adoption can grow, independent implementations can appear, and the developer ecosystem can compound. Graiphic's economic model belongs at the product, runtime, certification, support, and marketplace layers.

The intended direction is inspired by successful engine ecosystems such as Unreal Engine: reduce friction to adoption, let creators and companies build value, and monetize at the level of professional product experience, commercial success, marketplace distribution, certification, and ecosystem services rather than by closing the core language.

- the future proprietary Graiphic FROG IDE, with a commercial model oriented toward value creation rather than locking the language;
- professional runtime packages for selected platforms and deployment contexts;
- hardware bridge packages maintained by Graiphic or third-party vendors;
- compiler bridge packages and optimized execution corridors;
- certified extensions and conformance-verified packages;
- commercial support, enterprise services, and certification programs;
- a future FROG Store inside the Graiphic FROG IDE for toolkits, runtime families, compiler bridges, hardware profiles, widgets, domain packages, templates, and learning material.
- a future online shop or marketplace for extensions, runtimes, compiler bridges, hardware profiles, and domain-specific packages.

The FROG Store should also support free distribution. A third-party editor, hardware vendor, university lab, or open-source team should be able to publish free toolkits or free runtime packages. A commercial vendor should be able to choose its own licensing model. Graiphic

can take a commission on paid distribution through the platform while keeping the open specification available to everyone.

At the specification level, the message is simple: the standard is free to study and implement. At the product level, Graiphic can define the commercial terms of the official IDE, professional runtimes, certified packages, and store distribution. This keeps the ecosystem open while giving Graiphic a credible business model.

The branding distinction is important. FROG is the name of the language and standard. Graiphic's future proprietary flagship IDE will be the only commercial product allowed to carry the FROG name as its product name unless explicitly licensed. Other tools may be compatible with the FROG standard under their own brands.

This model is familiar to senior American technical and business readers: open standard, reference implementation, conformance program, developer ecosystem, marketplace, technical moat, platform leverage, and standards-based growth. The open language creates the adoption flywheel. The exclusive Graiphic IDE creates the flagship experience. Runtime families and the marketplace create ecosystem leverage.

17. Technology Surprise and Strategic Consequences

FROG should be understood as a potential technology surprise in the DARPA sense: not an incremental feature, but a shift in the structure of what becomes possible.

If FROG succeeds, the consequence is not merely that engineers get another graphical environment. The consequence is that graphical dataflow could move from closed product capability to open industrial infrastructure.

The advantages would be significant:

- engineering teams could inspect system logic graphically while preserving open machine-readable source;
- AI-assisted generation could be paired with human review and provenance tracking;
- hardware vendors could expose capability through runtimes and bridges without owning the language;
- compiler and runtime specialists could innovate downstream without redefining the source language;
- system integrators could select lighter or heavier runtimes depending on deployment constraints;
- critical industries could gain stronger audit trails from source to execution.

The disruptive consequence is therefore not visual programming alone. The disruptive consequence is open, modular, inspectable, AI-ready graphical system programming with a path toward heterogeneous hardware. That combination is rare, and it is exactly why FROG can become a serious platform strategy.

For historical graphical ecosystems, the risk is clear: if the AI era demands open provenance, inspectable IR, multi-runtime execution, multi-compiler pathways, runtime marketplaces, and hardware-agnostic bridges, then graphical platforms that remain too closed at the user ecosystem and technical ecosystem levels may lose relevance faster than expected. FROG is Graiphic's attempt to anticipate that paradigm shift rather than react to it too late.

18. Path Forward

1. Stabilize the core language surfaces: Expression, Language, IR, Libraries, Profiles, IDE-facing concepts, Conformance, Versioning, and Governance.
2. Protect Example 05 as the current bounded corridor and grow new examples around it rather than repeatedly redefining it.
3. Deepen runtime-family maturity across Python, Rust, and C/C++ while keeping the runtime downstream from language truth.
4. Deepen compiler-corridor proofs, including LLVM-oriented paths, without letting LLVM define FROG.
5. Port the SOTA and GO HW technology layers into FROG progressively: model execution, deep learning graph execution, inference and training orchestration, preprocessing and postprocessing graphs, hardware primitives, device profiles, target deployment, energy metrics, and runtime supervision.
6. Formalize runtime recipes for pure operation, observability, IDE development mode, hardware-vendor bridge mode, and distributed runtime/front-panel models.
7. Define marketplace, certification, branding, and commercial extension policies separately from the open specification.
8. Prepare IDE-level AI provenance and review mechanisms: colors, badges, Assisted-by metadata, validation status, conformance status, review checkpoints, and human acceptance markers.
9. Build targeted hardware bridge proofs for meaningful industrial targets without overclaiming universal hardware support.

The long-term exam is clear: FROG must prove that graphical system programming can be open, inspectable, modular, AI-ready, hardware-agnostic, compiler-agnostic, industrially serious, and economically viable without collapsing into toy tooling or vendor lock-in.

Conclusion

FROG is the continuation of GO HW on stronger ground. It is the path that allows the GO HW ambition to keep moving forward without depending on a closed third-party graphical language substrate.

SOTA GO showed that the graph can become the source of truth for AI workflows. It is operational, in production, and still under continuous development. GO HW showed the next technological direction: graph logic reaching systems, hardware, monitoring, deployment, and energy-aware execution. GO GenAI and GO IML extend the same programmatic vision toward generative orchestration and informed learning. These layers are currently strategic whitepaper directions rather than standalone completed developments.

FROG is the active development foundation that can progressively absorb and generalize those ambitions. It carries the same graph-orchestration logic into an open graphical language stack where system behavior can be expressed through .frog source, validated meaning, open FIR, lowering, backend contracts, runtime families, compiler corridors, hardware bridges, widgets, front panels, and observability surfaces.

This is the challenge Graiphic is choosing: not merely to build another graphical tool, but to open a new category of industrial programming infrastructure.

If successful, FROG can create several disruptions at once. It can make AI-generated software more inspectable by turning machine-generated .frog source into human-readable graphical dataflow architecture. It can accelerate development by allowing engineers and AI coding agents to co-produce structured programs that remain reviewable, validated, and traceable. It can restore human control through a cockpit where software structure, execution state, provenance, runtime health, hardware binding, observability, and front-panel interaction converge.

FROG can also make graphical programming hardware-agnostic by design. Through open FIR, runtime families, compiler corridors, and hardware bridges, the same language foundation can target different platforms with different operational strategies. Over time, this architecture can host the capabilities described by GO HW: sensing, inference, control, timing, communication, monitoring, energy awareness, deployment, and supervision.

FROG also generalizes access to graphical programming itself. By publishing an open specification, Graiphic creates the conditions for an ecosystem: universities can teach it, researchers can extend it, hardware vendors can target it, runtime specialists can implement it, and third-party developers can build widgets, toolkits, compiler bridges, hardware profiles, and domain-specific packages.

The future Graiphic FROG IDE is intended to support that adoption through a “free first, fee later if business” model inspired by modern engine ecosystems such as Unreal Engine. The goal is to reduce friction for students, researchers, independent developers, and industrial adopters, while building a professional ecosystem through the IDE, marketplace, certified extensions, runtimes, support, and commercial services.

FROG is therefore not only a technical answer. It is a response to a change in engineering culture. The next generation of engineers will not only write syntax; they will specify, generate, inspect, validate, orchestrate, and govern systems. FROG is built for that new role.

FROG does not claim every target is already supported. It does not claim graphics automatically guarantee security. It does not claim the flagship IDE already exists. Its claim is more credible and more consequential: the architecture needed for a new open category is now being defined and progressively implemented.

The future of AI-assisted industrial software will not be won only by generating more code. It will be won by the teams that can inspect, understand, govern, execute, and evolve what is generated.

FROG is built for that future.

The language stays open.
The execution becomes modular.
The ecosystem becomes extensible.
The cockpit becomes human.
And graphical dataflow becomes a candidate foundation for the next era of industrial software.

The language stays upstream. Execution becomes modular downstream. Humans regain control through inspectable graphical structure.

Appendix A - Technical Definitions

Term	Meaning
Canonical .frog source	The authoritative saved program artifact. It is open, structured, and intended to remain machine-friendly and reviewable.
Validated program meaning	The semantic layer that defines what a structurally valid FROG program means before execution-facing derivation.
FIR	FROG Intermediate Representation: the canonical open execution-facing representation derived from validated meaning.
Lowering	The transformation path from FIR toward backend-oriented forms without letting downstream execution define the language.
Backend contract	The explicit handoff boundary between open upstream language artifacts and downstream runtime, compiler, or hardware families.
Runtime	The operational execution layer: scheduling, state, widgets, IO, monitoring, observability, communication, debugging, and interaction depending on the runtime family.
Compiler	The transformation chain that prepares target-specific executable or deployable artifacts.
Runtime family	A class of runtime implementations with a defined operating posture, such as operational-only, observability-enabled, development, distributed, or hardware-vendor bridge runtime.
Front panel	A UI-facing layer for human interaction, monitoring, control, and observability. It does not replace diagram authority.
Graiphic FROG IDE	Graiphic's future flagship proprietary IDE built on the open FROG standard and intended to be the only commercial product carrying the FROG product name.

Appendix B - Source and Evidence Trail

The following public sources were used to align this whitepaper with the public FROG repository, the GO whitepaper trajectory, NI/LabVIEW context, and the AI contribution governance landscape.

- FROG repository root: https://github.com/Graiphic/FROG/tree/main
- FROG IR architecture: https://github.com/Graiphic/FROG/blob/main/IR/Readme.md
- FROG Example 05 bounded UI accumulator: https://github.com/Graiphic/FROG/blob/main/Examples/05_bounded_ui_accumulator/Readme.md
- FROG reference runtime family: https://github.com/Graiphic/FROG/blob/main/Implementations/Reference/Runtime/Readme.md
- FROG strategy: https://github.com/Graiphic/FROG/blob/main/Strategy/Readme.md
- FROG governance: https://github.com/Graiphic/FROG/blob/main/GOVERNANCE.md
- FROG versioning: https://github.com/Graiphic/FROG/blob/main/Versioning/Readme.md
- Graiphic GO Whitepaper Series: https://github.com/Graiphic/GO-Whitepapers
- Graiphic GO Whitepaper GitHub Pages: https://graiphic.github.io/GO-Whitepapers/
- SOTA GO page: https://github.com/Graiphic/GO-Whitepapers/tree/main/SOTA%20GO
- GO HW page: https://github.com/Graiphic/GO-Whitepapers/tree/main/GO%20HW%20%E2%80%94%20From%20Models%20to%20Systems
- NI LabVIEW Compiler: Under the Hood: https://www.ni.com/en/support/documentation/supplemental/10/ni-labview-compiler--under-the-hood.html
- Linux kernel AI coding assistants: https://docs.kernel.org/process/coding-assistants.html
- QEMU code provenance: https://www.qemu.org/docs/master/devel/code-provenance.html
- NetBSD commit guidelines: https://www.netbsd.org/developers/commit-guidelines.html
- Gentoo Council AI policy: https://wiki.gentoo.org/wiki/Project:Council/AI_policy
- Servo contribution policy: https://book.servo.org/contributing/getting-started.html
- elementary OS generative AI policy: https://docs.elementary.io/contributor-guide/development/generative-ai-policy
- GIMP developer internship policy: https://developer.gimp.org/core/internship/
- GNOME Shell Extensions review guidelines: https://gjs.guide/extensions/review-guidelines/review-guidelines.html
- Daniel Stenberg - The end of the curl bug-bounty: https://daniel.haxx.se/blog/2026/01/26/the-end-of-the-curl-bug-bounty/
- Unreal Engine licensing overview: https://www.unrealengine.com/license
- Epic Games Store revenue share program: https://store.epicgames.com/en-US/distribution/revenue-programs/revenue-share
- Epic Games Store distribution agreement: https://dev.epicgames.com/portal/en-US/distribution-agreement